

个1组成。这些位模式有助于掩码运算。这种模式能够通过C表达式 $(1 \ll k) - 1$ 生成, 利用的是这样一个属性, 即我们想要的位模式的数值为 $2^k - 1$ 。例如, 表达式 $(1 \ll 8) - 1$ 将产生位模式 0xFF。

浮点表示通过将数字编码为 $x \times 2^y$ 的形式来近似地表示实数。最常见的浮点表示方式是由 IEEE 标准 754 定义的。它提供了几种不同的精度, 最常见的是单精度(32 位)和双精度(64 位)。IEEE 浮点也能够表示特殊值 $+\infty$ 、 $-\infty$ 和 NaN。

必须非常小心地使用浮点运算, 因为浮点运算只有有限的范围和精度, 而且并不遵守普遍的算术属性, 比如结合性。

参考文献说明

关于C语言的参考书[45, 61]讨论了不同的数据类型和运算的属性。(这两本书中, 只有 Steele 和 Harbison 的书[45]涵盖了 ISO C99 中的新特性。目前还没有看到任何涉及 ISO C11 新特性的书籍。)对于精确的字长或者数字编码 C 语言标准没有详细的定义。这些细节是故意省去的, 这样可以在更大范围的不同机器上实现 C 语言。已经有几本书[59, 74]给了 C 语言程序员一些建议, 警告他们关于溢出、隐式强制类型转换到无符号数, 以及其他一些已经在这一章中谈及的陷阱。这些书还提供了对变量命名、编码风格和代码测试的有益建议。Seacord 的书[97]是关于 C 和 C++ 程序中的安全问题的, 本书结合了 C 程序的有关信息, 介绍了如何编译和执行程序, 以及漏洞是如何造成的。关于 Java 的书(我们推荐 Java 语言的创始人 James Gosling 参与编写的一本书[5])描述了 Java 支持的数据格式和算术运算。

关于逻辑设计的书[58, 116]都有关于编码和算术运算的章节, 描述了实现算术电路的不同方式。Overton 的关于 IEEE 浮点数的书[82], 从数字应用程序员的角度, 详细描述了格式和属性。

家庭作业

- 2.55 在你能够访问的不同机器上, 使用 show_bytes(文件 show-bytes.c)编译并运行示例代码。确定这些机器使用的字节顺序。
- 2.56 试着用不同的示例值来运行 show_bytes 的代码。
- 2.57 编写程序 show_short、show_long 和 show_double, 它们分别打印类型为 short、long 和 double 的 C 语言对象的字节表示。请试着在几种机器上运行。
- 2.58 编写过程 is_little_endian, 当在小端法机器上编译和运行时返回 1, 在大端法机器上编译运行时则返回 0。这个程序应该可以运行在任何机器上, 无论机器的字长是多少。
- 2.59 编写一个 C 表达式, 它生成一个字, 由 x 的最低有效字节和 y 中剩下的字节组成。对于运算数 $x = 0x89ABCDEF$ 和 $y = 0x76543210$, 就得到 0x765432EF。
- 2.60 假设我们将一个 w 位的字中的字节从 0(最低位)到 $w/8 - 1$ (最高位)编号。写出下面 C 函数的代码, 它会返回一个无符号值, 其中参数 x 的字节 i 被替换成字节 b:

```
unsigned replace_byte(unsigned x, int i, unsigned char b);
```

以下示例, 说明了这个函数该如何工作:

```
replace_byte(0x12345678, 2, 0xAB) --> 0x12AB5678
replace_byte(0x12345678, 0, 0xAB) --> 0x123456AB
```

位级整数编码规则

在接下来的作业中, 我们特意限制了你能使用的编程结构, 来帮你更好地理解 C 语言的位级、逻辑和算术运算。在回答这些问题时, 你的代码必须遵守以下规则:

- 假设
 - 整数用补码形式表示。
 - 有符号数的右移是算术右移。
 - 数据类型 int 是 w 位长的。对于某些题目, 会给定 w 的值, 但是在其他情况下, 只要 w 是 8 的整数倍, 你的代码就应该能工作。你可以用表达式 $\text{sizeof(int)} \ll 3$ 来计算 w。

- 禁止使用

- 条件语句(if 或者?:)、循环、分支语句、函数调用和宏调用。
- 除法、模运算和乘法。
- 相对比较运算(<、>、<=和>=)。

- 允许的运算

- 所有的位级和逻辑运算。
- 左移和右移,但是位移量只能在 0 和 $w-1$ 之间。
- 加法和减法。
- 相等(==)和不相等(!=)测试。(在有些题目中,也不允许这些运算。)
- 整型常数 INT_MIN 和 INT_MAX。
- 对 int 和 unsigned 进行强制类型转换,无论是显式的还是隐式的。

即使有这些条件的限制,你仍然可以选择带有描述性的变量名,并且使用注释来描述你的解决方案的逻辑,尽量提高代码的可读性。例如,下面这段代码从整数参数 x 中抽取出最高有效字节:

```
/* Get most significant byte from x */
int get_msb(int x) {
    /* Shift by w-8 */
    int shift_val = (sizeof(int)-1)<<3;
    /* Arithmetic shift */
    int xright = x >> shift_val;
    /* Zero all but LSB */
    return xright & 0xFF;
}
```

2.61 写一个 C 表达式,在下列描述的条件下产生 1,而在其他情况下得到 0。假设 x 是 int 类型。

- x 的任何位都等于 1。
- x 的任何位都等于 0。
- x 的最低有效字节中的位都等于 1。
- x 的最高有效字节中的位都等于 0。

代码应该遵循位级整数编码规则,另外还有一个限制,你不能使用相等(==)和不相等(!=)测试。

2.62 编写一个函数 int_shifts_are_arithmetic(),在对 int 类型的数使用算术右移的机器上运行时这个函数生成 1,而其他情况下生成 0。你的代码应该可以运行在任何字长的机器上。在几种机器上测试你的代码。

2.63 将下面的 C 函数代码补充完整。函数 srl 用算术右移(由值 xsra 给出)来完成逻辑右移,后面的其他操作不包括右移或者除法。函数 sra 用逻辑右移(由值 xsrl 给出)来完成算术右移,后面的其他操作不包括右移或者除法。可以通过计算 $8 * \text{sizeof}(\text{int})$ 来确定数据类型 int 中的位数 w 。位移量 k 的取值范围为 $0 \sim w-1$ 。

```
unsigned srl(unsigned x, int k) {
    /* Perform shift arithmetically */
    unsigned xsra = (int) x >> k;
    :
    :
    :
    :
}

int sra(int x, int k) {
    /* Perform shift logically */
    int xsrl = (unsigned) x >> k;
    :
    :
    :
    :
}
```


* 2.64 写出代码实现如下函数：

```
/* Return 1 when any odd bit of x equals 1; 0 otherwise.
   Assume w=32 */
int any_odd_one(unsigned x);
```

函数应该遵循位级整数编码规则，不过你可以假设数据类型 `int` 有 $w=32$ 位。

** 2.65 写出代码实现如下函数：

```
/* Return 1 when x contains an odd number of 1s; 0 otherwise.
   Assume w=32 */
int odd_ones(unsigned x);
```

函数应该遵循位级整数编码规则，不过你可以假设数据类型 `int` 有 $w=32$ 位。

你的代码最多只能包含 12 个算术运算、位运算和逻辑运算。

** 2.66 写出代码实现如下函数：

```
/*
 * Generate mask indicating leftmost 1 in x. Assume w=32.
 * For example, 0xFF00 -> 0x8000, and 0x6600 -> 0x4000.
 * If x = 0, then return 0.
 */
int leftmost_one(unsigned x);
```

函数应该遵循位级整数编码规则，不过你可以假设数据类型 `int` 有 $w=32$ 位。

你的代码最多只能包含 15 个算术运算、位运算和逻辑运算。

提示：先将 x 转换成形如 $[0 \cdots 011 \cdots 1]$ 的位向量。

** 2.67 给你一个任务，编写一个过程 `int_size_is_32()`，当在一个 `int` 是 32 位的机器上运行时，该程序产生 1，而其他情况则产生 0。不允许使用 `sizeof` 运算符。下面是开始时的尝试：

```
1  /* The following code does not run properly on some machines */
2  int bad_int_size_is_32() {
3      /* Set most significant bit (msb) of 32-bit machine */
4      int set_msb = 1 << 31;
5      /* Shift past msb of 32-bit word */
6      int beyond_msb = 1 << 32;
7
8      /* set_msb is nonzero when word size >= 32
9       beyond_msb is zero when word size <= 32 */
10     return set_msb && !beyond_msb;
11 }
```

当在 SUN SPARC 这样的 32 位机器上编译并运行时，这个过程返回的却是 0。下面的编译器信息给了我们一个问题的指示：

```
warning: left shift count >= width of type
```

- 我们的代码在哪个方面没有遵守 C 语言标准？
- 修改代码，使得它在 `int` 至少为 32 位的任何机器上都能正确地运行。
- 修改代码，使得它在 `int` 至少为 16 位的任何机器上都能正确地运行。

** 2.68 写出具有如下原型的函数的代码：

```
/*
 * Mask with least significant n bits set to 1
 * Examples: n = 6 --> 0x3F, n = 17 --> 0x1FFFF
 * Assume 1 <= n <= w
 */
int lower_one_mask(int n);
```

函数应该遵循位级整数编码规则。要注意 $n=w$ 的情况。

** 2.69 写出具有如下原型的函数的代码：

```

/*
 * Do rotating left shift. Assume  $0 \leq n < w$ 
 * Examples when  $x = 0x12345678$  and  $w = 32$ :
 *    $n=4 \rightarrow 0x23456781$ ,  $n=20 \rightarrow 0x67812345$ 
 */
unsigned rotate_left(unsigned x, int n);

```

函数应该遵循位级整数编码规则。要注意 $n=0$ 的情况。

••2.70 写出具有如下原型的函数的代码：

```

/*
 * Return 1 when x can be represented as an n-bit, 2's-complement
 * number; 0 otherwise
 * Assume  $1 \leq n \leq w$ 
 */
int fits_bits(int x, int n);

```

函数应该遵循位级整数编码规则。

•2.71 你刚刚开始开始在一家公司工作，他们要实现一组过程来操作一个数据结构，要将4个有符号字节封装成一个32位 unsigned。一个字中的字节从0(最低有效字节)编号到3(最高有效字节)。分配给你的任务是：为一个使用补码运算和算术右移的机器编写一个具有如下原型的函数：

```

/* Declaration of data type where 4 bytes are packed
   into an unsigned */
typedef unsigned packed_t;

/* Extract byte from word. Return as signed integer */
int xbyte(packed_t word, int bytenum);

```

也就是说，函数会抽取指定的字节，再把它符号扩展为一个32位 int。

你的前任(因为水平不够高而被解雇了)编写了下面的代码：

```

/* Failed attempt at xbyte */
int xbyte(packed_t word, int bytenum)
{
    return (word >> (bytenum << 3)) & 0xFF;
}

```

A. 这段代码错在哪里？

B. 给出函数的正确实现，只能使用左右移位和一个减法。

••2.72 给你一个任务，写一个函数，将整数 val 复制到缓冲区 buf 中，但是只有当缓冲区中有足够可用的空间时，才执行复制。

你写的代码如下：

```

/* Copy integer into buffer if space is available */
/* WARNING: The following code is buggy */
void copy_int(int val, void *buf, int maxbytes) {
    if (maxbytes - sizeof(val) >= 0)
        memcpy(buf, (void *) &val, sizeof(val));
}

```

这段代码使用了库函数 memcpy。虽然在这里用这个函数有点刻意，因为我们只是想复制一个 int，但是它说明了一种复制较大数据结构的常见方法。

你仔细地测试了这段代码后发现，哪怕 maxbytes 很小的时候，它也能把值复制到缓冲区中。

A. 解释为什么代码中的条件测试总是成功。提示：sizeof 运算符返回类型为 size_t 的值。

B. 你该如何重写这个条件测试，使之工作正确。

••2.73 写出具有如下原型的函数的代码：

```

/* Addition that saturates to TMin or TMax */
int saturating_add(int x, int y);

```


同正常的补码加法溢出的方式不同，当正溢出时，饱和加法返回 $TM_{\max}$ ，负溢出时，返回 $TM_{\min}$ 。饱和运算常常用在执行数字信号处理的程序中。

你的函数应该遵循位级整数编码规则。

- 2.74 写出具有如下原型的函数的代码：

```
/* Determine whether arguments can be subtracted without overflow */
int tsub_ok(int x, int y);
```

如果计算 $x-y$ 不溢出，这个函数就返回 1。

- 2.75 假设我们想要计算 $x \cdot y$ 的完整的 $2w$ 位表示，其中， x 和 y 都是无符号数，并且运行在数据类型 `unsigned` 是 w 位的机器上。乘积的低 w 位能够用表达式 $x*y$ 计算，所以，我们只需要一个具有下列原型的函数：

```
unsigned unsigned_high_prod(unsigned x, unsigned y);
```

这个函数计算无符号变量 $x \cdot y$ 的高 w 位。

我们使用一个具有下面原型的库函数：

```
int signed_high_prod(int x, int y);
```

它计算在 x 和 y 采用补码形式的情况下， $x \cdot y$ 的高 w 位。编写代码调用这个过程，以实现用无符号数为参数的函数。验证你的解答的正确性。

提示：看看等式(2.18)的推导中，有符号乘积 $x \cdot y$ 和无符号乘积 $x' \cdot y'$ 之间的关系。

- 2.76 库函数 `calloc` 有如下声明：

```
void *calloc(size_t nmemb, size_t size);
```

根据库文档：“函数 `calloc` 为一个数组分配内存，该数组有 `nmemb` 个元素，每个元素为 `size` 字节。内存设置为 0。如果 `nmemb` 或 `size` 为 0，则 `calloc` 返回 `NULL`。”

编写 `calloc` 的实现，通过调用 `malloc` 执行分配，调用 `memset` 将内存设置为 0。你的代码应该没有任何由算术溢出引起的漏洞，且无论数据类型 `size_t` 用多少位表示，代码都应该正常工作。

作为参考，函数 `malloc` 和 `memset` 声明如下：

```
void *malloc(size_t size);
void *memset(void *s, int c, size_t n);
```

- 2.77 假设我们有一个任务：生成一段代码，将整数变量 x 乘以不同的常数因子 K 。为了提高效率，我们想只使用 $+$ 、 $-$ 和 $\ll$ 运算。对于下列 K 的值，写出执行乘法运算的 C 表达式，每个表达式中最多使用 3 个运算。

- A. $K=17$
- B. $K=-7$
- C. $K=60$
- D. $K=-112$

- 2.78 写出具有如下原型的函数的代码：

```
/* Divide by power of 2. Assume  $0 \leq k < w-1$  */
int divide_power2(int x, int k);
```

该函数要用正确的舍入方式计算 $x/2^k$ ，并且应该遵循位级整数编码规则。

- 2.79 写出函数 `mul3div4` 的代码，对于整数参数 x ，计算 $3*x/4$ ，但是要遵循位级整数编码规则。你的代码计算 $3*x$ 也会产生溢出。

- 2.80 写出函数 `threefourths` 的代码，对于整数参数 x ，计算 $3/4x$ 的值，向零舍入。它不会溢出。函数应该遵循位级整数编码规则。

- 2.81 编写 C 表达式产生如下位模式，其中 a^k 表示符号 a 重复 k 次。假设一个 w 位的数据类型。代码可以包含对参数 j 和 k 的引用，它们分别表示 j 和 k 的值，但是不能使用表示 w 的参数。

A. $1^{w-k}0^k$ B. $0^{w-k-j}1^k0^j$

- 2.82 我们在一个 int 类型值为 32 位的机器上运行程序。这些值以补码形式表示，而且它们都是算术右移的。unsigned 类型的值也是 32 位的。

我们产生随机数 x 和 y ，并且把它们转换成无符号数，显示如下：

```
/* Create some arbitrary values */
int x = random();
int y = random();
/* Convert to unsigned */
unsigned ux = (unsigned) x;
unsigned uy = (unsigned) y;
```

对于下列每个 C 表达式，你要指出表达式是否总是为 1。如果它总是为 1，那么请描述其中的数学原理。否则，列举出一个使它为 0 的参数示例。

A. $(x < y) == (-x > -y)$ B. $((x+y) < 4) + y - x == 17*y + 15*x$ C. $\sim x + \sim y + 1 == \sim (x+y)$ D. $(ux - uy) == -(\text{unsigned})(y - x)$ E. $((x > 2) < 2) <= x$

- 2.83 一些数字的二进制表示是由形如 $0.yyy\ldots$ 的无穷串组成的，其中 y 是一个 k 位的序列。例如， $\frac{1}{3}$ 的二进制表示是 $0.01010101\ldots$ ($y = 01$)，而 $\frac{1}{5}$ 的二进制表示是 $0.001100110011\ldots$ ($y = 0011$)。

A. 设 $Y = B2U_k(y)$ ，也就是说，这个数具有二进制表示 y 。给出一个由 Y 和 k 组成的公式表示这个无穷串的值。

提示：请考虑将二进制小数点右移 k 位的结果。

B. 对于下列的 y 值，串的值是多少？

(a) 101

(b) 0110

(c) 010011

- 2.84 填写下列程序的返回值，这个程序测试它的第一个参数是否小于或者等于第二个参数。假定函数 $f2u$ 返回一个无符号 32 位数字，其位表示与它的浮点参数相同。你可以假设两个参数都不是 NaN。两种 0，+0 和 -0 被认为是相等的。

```
int float_le(float x, float y) {
    unsigned ux = f2u(x);
    unsigned uy = f2u(y);

    /* Get the sign bits */
    unsigned sx = ux >> 31;
    unsigned sy = uy >> 31;

    /* Give an expression using only ux, uy, sx, and sy */
    return _____;
}
```

- 2.85 给定一个浮点格式，有 k 位指数和 n 位小数，对于下列数，写出阶码 E 、尾数 M 、小数 f 和值 V 的公式。另外，请描述其位表示。

A. 数 7.0。

B. 能够被准确描述的最大奇整数。

C. 最小的规格化数的倒数。

- 2.86 与 Intel 兼容的处理器也支持“扩展精度”浮点形式，这种格式具有 80 位字长，被分成 1 个符号

位、 $k=15$ 个阶码位、1 个单独的整数位和 $n=63$ 个小数位。整数位是 IEEE 浮点表示中隐含位的显式副本。也就是说，对于规格化的值它等于 1，对于非规格化的值它等于 0。填写下表，给出用这种格式表示的一些“有趣的”数字的近似值。

描 述	扩展精度	
	值	十进制
最小的正非规格化数		
最小的正规格化数		
最大的规格化数		

将数据类型声明为 long double，就可以把这种格式用于为与 Intel 兼容的机器编译 C 程序。但是，它会强制编译器以传统的 8087 浮点指令为基础生成代码。由此产生的程序很可能会比数据类型为 float 或 double 的情况慢上许多。

• 2.87 2008 版 IEEE 浮点标准，即 IEEE 754-2008，包含了一种 16 位的“半精度”浮点格式。它最初是由计算机图形公司设计的，其存储的数据所需的动态范围要高于 16 位整数可获得的范围。这种格式具有 1 个符号位、5 个阶码位($k=5$)和 10 个小数位($n=10$)。阶码偏置量是 $2^{5-1}-1=15$ 。

对于每个给定的数，填写下表，其中，每一列具有如下指示说明：

Hex: 描述编码形式的 4 个十六进制数字。

M: 尾数的值。这应该是一个形如 x 或 $\frac{x}{y}$ 的数，其中 x 是一个整数，而 y 是 2 的整数幂。例

如: 0、 $\frac{67}{64}$ 和 $\frac{1}{256}$ 。

E: 阶码的整数值。

V: 所表示的数字值。使用 x 或者 $x \times 2^z$ 表示，其中 x 和 z 都是整数。

D: (可能近似的)数值，用 printf 的格式规范 %f 打印。

举一个例子，为了表示数 $\frac{7}{8}$ ，我们有 $s=0$ ， $M=\frac{7}{4}$ 和 $E=-1$ 。因此这个数的阶码字段为 01110_2 (十进制值 $15-1=14$)，尾数字段为 1100000000_2 ，得到一个十六进制的表示 3B00。其数值为 0.875。

标记为“—”的条目不用填写。

描 述	Hex	M	E	V	D
-0				-0	-0.0
最小的 >2 的值					
512				512	512.0
最大的非规格化数					
$-\infty$		—	—	$-\infty$	$-\infty$
十六进制表示为 3BB0 的数	3BB0				

•• 2.88 考虑下面两个基于 IEEE 浮点格式的 9 位浮点表示。

1. 格式 A

- 有一个符号位。
- 有 $k=5$ 个阶码位。阶码偏置量是 15。
- 有 $n=3$ 个小数位。

2. 格式 B

- 有一个符号位。
- 有 $k=4$ 个阶码位。阶码偏置量是 7。

- 有 $n=4$ 个小数位。

下面给出了一些格式 A 表示的位模式，你的任务是把它们转换成最接近的格式 B 表示的值。如果需要舍入，你要向 $+\infty$ 舍入。另外，给出用格式 A 和格式 B 表示的位模式对应的值。要么是整数(例如 17)，要么是小数(例如 $17/64$ 或 $17/2^6$)。

格式A		格式B	
位	值	位	值
1 01110 001	$-\frac{9}{16}$	1 0110 0010	$-\frac{9}{16}$
0 10110 101			
1 00111 110			
0 00000 101			
1 11011 000			
0 11000 100			

- 2.89 我们在一个 int 类型为 32 位补码表示的机器上运行程序。float 类型的值使用 32 位 IEEE 格式，而 double 类型的值使用 64 位 IEEE 格式。

我们产生随机整数 x、y 和 z，并且把它们转换成 double 类型的值：

```
/* Create some arbitrary values */
int x = random();
int y = random();
int z = random();
/* Convert to double */
double dx = (double) x;
double dy = (double) y;
double dz = (double) z;
```

对于下列的每个 C 表达式，你要指出表达式是否总是为 1。如果它总是为 1，描述其中的数学原理。否则，列举出使它为 0 的参数的例子。请注意，不能使用 IA32 机器运行 GCC 来测试你的答案，因为对于 float 和 double，它使用的都是 80 位的扩展精度表示。

- A. (float)x==(float)dx
- B. dx-dy==(double) (x-y)
- C. (dx+dy)+dz==dx+(dy+dz)
- D. (dx*dy)*dz==dx*(dy*dz)
- E. dx/dx==dz/dz

- 2.90 分配给你一个任务，编写一个 C 函数来计算 2^x 的浮点表示。你意识到完成这个任务的最好方法是直接创建结果的 IEEE 单精度表示。当 x 太小时，你的程序将返回 0.0。当 x 太大时，它会返回 $+\infty$ 。填写下列代码的空白部分，以计算出正确的结果。假设函数 u2f 返回的浮点值与它的无符号参数有相同的位表示。

```
float fpwr2(int x)
{
    /* Result exponent and fraction */
    unsigned exp, frac;
    unsigned u;

    if (x < _____) {
        /* Too small. Return 0.0 */
        exp = _____;
        frac = _____;
    } else if (x < _____) {
        /* Denormalized result */
        exp = _____;
        frac = _____;
```



```

    } else if (x < _____) {
        /* Normalized result. */
        exp = _____;
        frac = _____;
    } else {
        /* Too big. Return +oo */
        exp = _____;
        frac = _____;
    }

    /* Pack exp and frac into 32 bits */
    u = exp << 23 | frac;
    /* Return as float */
    return u2f(u);
}

```

- * 2.91 大约公元前 250 年，希腊数学家阿基米德证明了 $\frac{223}{71} < \pi < \frac{22}{7}$ 。如果当时有一台计算机和标准库 `<math.h>`，他就能确定 π 的单精度浮点近似值的十六进制表示为 `0x40490FDB`。当然，所有的这些都只是近似值，因为 π 不是有理数。
- A. 这个浮点值表示的二进制小数是多少？
- B. $\frac{22}{7}$ 的二进制小数表示是什么？提示：参见家庭作业 2.83。
- C. 这两个 π 的近似值从哪一位（相对于二进制小数点）开始不同的？

位级浮点编码规则

在接下来的题目中，你所写的代码要实现浮点函数在浮点数的位级表示上直接运算。你的代码应该完全遵循 IEEE 浮点运算的规则，包括当需要舍入时，要使用向偶数舍入的方式。

为此，我们把数据类型 `float_bits` 等价于 `unsigned`：

```

/* Access bit-level representation floating-point number */
typedef unsigned float_bits;

```

你的代码中不使用数据类型 `float`，而要使用 `float_bits`。你可以使用数据类型 `int` 和 `unsigned`，包括无符号和整数常数和运算。你不可以使用任何联合、结构和数组。更重要的是，你不能使用任何浮点数据类型、运算或者常数。取而代之，你的代码应该执行实现这些指定的浮点运算的位操作。

下面的函数说明了对这些规则的使用。对于参数 f ，如果 f 是非规格化的，该函数返回 ± 0 （保持 f 的符号），否则，返回 f 。

```

/* If f is denorm, return 0. Otherwise, return f */
float_bits float_denorm_zero(float_bits f) {
    /* Decompose bit representation into parts */
    unsigned sign = f >> 31;
    unsigned exp = f >> 23 & 0xFF;
    unsigned frac = f & 0x7FFFFF;
    if (exp == 0) {
        /* Denormalized. Set fraction to 0 */
        frac = 0;
    }
    /* Reassemble bits */
    return (sign << 31) | (exp << 23) | frac;
}

```

- ** 2.92 遵循位级浮点编码规则，实现具有如下原型的函数：

```

/* Compute -f. If f is NaN, then return f. */
float_bits float_negate(float_bits f);

```

对于浮点数 f ，这个函数计算 $-f$ 。如果 f 是 `NaN`，你的函数应该简单地返回 f 。

测试你的函数，对参数 f 可以取的所有 2^{32} 个值求值，将结果与你使用机器的浮点运算得到的结果

相比较。

- 2.93 遵循位级浮点编码规则，实现具有如下原型的函数：

```
/* Compute |f|. If f is NaN, then return f. */
float_bits float_absval(float_bits f);
```

对于浮点数 f ，这个函数计算 $|f|$ 。如果 f 是 NaN ，你的函数应该简单地返回 f 。

测试你的函数，对参数 f 可以取的所有 2^{32} 个值求值，将结果与你使用机器的浮点运算得到的结果相比较。

- 2.94 遵循位级浮点编码规则，实现具有如下原型的函数：

```
/* Compute 2*f. If f is NaN, then return f. */
float_bits float_twice(float_bits f);
```

对于浮点数 f ，这个函数计算 $2.0 \cdot f$ 。如果 f 是 NaN ，你的函数应该简单地返回 f 。

测试你的函数，对参数 f 可以取的所有 2^{32} 个值求值，将结果与你使用机器的浮点运算得到的结果相比较。

- 2.95 遵循位级浮点编码规则，实现具有如下原型的函数：

```
/* Compute 0.5*f. If f is NaN, then return f. */
float_bits float_half(float_bits f);
```

对于浮点数 f ，这个函数计算 $0.5 \cdot f$ 。如果 f 是 NaN ，你的函数应该简单地返回 f 。

测试你的函数，对参数 f 可以取的所有 2^{32} 个值求值，将结果与你使用机器的浮点运算得到的结果相比较。

- 2.96 遵循位级浮点编码规则，实现具有如下原型的函数：

```
/*
 * Compute (int) f.
 * If conversion causes overflow or f is NaN, return 0x80000000
 */
int float_f2i(float_bits f);
```

对于浮点数 f ，这个函数计算 $(\text{int})f$ 。如果 f 是 NaN ，你的函数应该向零舍入。如果 f 不能用整数表示（例如，超出表示范围，或者它是一个 NaN ），那么函数应该返回 $0x80000000$ 。

测试你的函数，对参数 f 可以取的所有 2^{32} 个值求值，将结果与你使用机器的浮点运算得到的结果相比较。

- 2.97 遵循位级浮点编码规则，实现具有如下原型的函数：

```
/* Compute (float) i */
float_bits float_i2f(int i);
```

对于函数 i ，这个函数计算 $(\text{float}) i$ 的位级表示。

测试你的函数，对参数 f 可以取的所有 2^{32} 个值求值，将结果与你使用机器的浮点运算得到的结果相比较。

练习题答案

- 2.1 在我们开始查看机器级程序的时候，理解十六进制和二进制格式之间的关系将是很重要的。虽然本书中介绍了完成这些转换的方法，但是做点练习能够让你更加熟练。

- A. 将 $0x39A7F8$ 转换成二进制：

十六进制	3	9	A	7	F	8
二进制	0011	1001	1010	0111	1111	1000

- B. 将二进制 1100100101111011 转换成十六进制：

二进制	1100	1001	0111	1011
十六进制	C	9	7	B